Programming Fundamentals

Week 6 — Introducing the PC Troubleshooter

Programming Fundamentals Team

SETU

2026-10-20

Remember Assignment 1: Make My Decision!?

In Assignment 1, you built a program that **made a recommendation**.

Your program:

- asked the user for information
- used `if`, `elif` and `else` to check their answers
- displayed a recommendation

Remember Assignment 1: Make My Decision!?

In Assignment 1, you built a program that **made a recommendation**.

Your program:

- asked the user for information
- used `if`, `elif` and `else` to check their answers
- displayed a recommendation

You have already written the beginnings of a rule-based system!

What Did Your Rules Look Like?

A simple decision might have looked like this:

```
if budget < 500:
    print("Choose the budget option")
else:
    print("Consider the premium option")
```

The rule is **written directly into the program**.

What Did Your Rules Look Like?

A simple decision might have looked like this:

```
if budget < 500:
    print("Choose the budget option")
else:
    print("Consider the premium option")
```

The rule is **written directly into the program**.

Question: What would you need to change if you wanted to add 20 more rules?

Assignment 2: Build an Expert System

We are going to develop that idea further.

Instead of one small decision, our program will:

- ask a series of questions
- use the answers to check **rules**
- suggest possible solutions
- explain **why** a recommendation was made

The big change: we will store much of the expert knowledge as data, rather than hard-code every decision.

Our Worked Example: PC Troubleshooter

Imagine someone tells you:

“My computer isn’t working!”

You cannot recommend a solution yet. You need more information.

You might ask:

- Does the computer turn on?
- Is the power cable connected?
- Is the power indicator light on?
- Does the monitor show an image?

Our Worked Example: PC Troubleshooter

Imagine someone tells you:

“My computer isn’t working!”

You cannot recommend a solution yet. You need more information.

You might ask:

- Does the computer turn on?
- Is the power cable connected?
- Is the power indicator light on?
- Does the monitor show an image?

Discuss: Why is “My computer isn’t working” not enough information?

Why Do We Need Questions?

Questions collect evidence about the problem.

```
questions = [
    {"id": 1, "text": "Does the computer turn on?"},
    {"id": 2, "text": "Is the power cable connected?"},
    {"id": 3, "text": "Is the power light on?"}
]
```

Each question is a **dictionary**.

All the questions are stored in a **list**.

We already know how to loop through, search and update lists of dictionaries. .

Why Do We Need Recommendations?

A recommendation is the **advice** our system might give.

```
recommendations = [
    {"id": 1, "title": "Check the power cable"},
    {"id": 2, "title": "Check the power supply"},
    {"id": 3, "title": "Check the monitor"}
]
```

Notice that recommendations have IDs too.

Later, a rule can refer to a recommendation by its ID.

Why Do We Need Recommendations?

A recommendation is the **advice** our system might give.

```
recommendations = [
    {"id": 1, "title": "Check the power cable"},
    {"id": 2, "title": "Check the power supply"},
    {"id": 3, "title": "Check the monitor"}
]
```

Notice that recommendations have IDs too.

Later, a rule can refer to a recommendation by its ID.

Question: Is a recommendation the same thing as a question?

Why Do We Need Rules?

A **rule** connects the evidence to a recommendation.

IF the computer does not turn on, **THEN** suggest checking the power cable.

We can represent that simple rule as data:

```
rule = {
    "id": 1,
    "question_id": 1,
    "expected_answer": "no",
    "recommendation_id": 1
}
```

This is our first, simplified rule format. Later, rules can check several answers together.

From Assignment 1 to Assignment 2

Assignment 1	Assignment 2
Ask for input	Ask troubleshooting questions
Decisions written using `if` / `elif`	Rules stored as data
Display a recommendation	Match rules and explain recommendations
Small, fixed decision program	Knowledge that can grow and change

From Assignment 1 to Assignment 2

Assignment 1	Assignment 2
Ask for input	Ask troubleshooting questions
Decisions written using `if` / `elif`	Rules stored as data
Display a recommendation	Match rules and explain recommendations
Small, fixed decision program	Knowledge that can grow and change

The same programming ideas — but a more flexible design.

What Are We Building This Week?

Not the finished expert system!

This week, we will practise managing its data.

- Display the questions
- Find a question by ID
- Add a question
- Update a question
- Delete a question

These are the **CRUD** operations: Create, Read, Update, Delete.

> We'll start with lists, dictionaries and functions — no classes or JSON required yet.

Quick Class Activity

A user says: **“The screen is blank.”**

Write:

1. **Two questions** you would ask
2. **Two recommendations** you might give
3. **One IF/THEN rule** connecting an answer to advice

Quick Class Activity

A user says: **“The screen is blank.”**

Write:

1. **Two questions** you would ask
2. **Two recommendations** you might give
3. **One IF/THEN rule** connecting an answer to advice

Then consider: Which parts are **data**? Which part is the **decision**?

Class Activity: One Possible Answer

A user tells us: **"The screen is blank."**

Two questions we could ask:

1. Is the monitor switched on?
2. Is the monitor cable connected properly?

Two recommendations we could give:

1. Switch on the monitor and check its power connection.
2. Check that the cable between the monitor and computer is secure.

There are several valid answers. The important thing is to ask useful questions and suggest relevant advice.

Connecting a Question to a Recommendation

Our first IF/THEN rule:

> **IF** the monitor is not switched on, **THEN** recommend switching it on.

The rule connects three things:

- **Question:** Is the monitor switched on?
- **Answer:** No
- **Recommendation:** Switch on the monitor.

Would we give the same recommendation if the answer was "Yes"?

Representing the Questions and Advice as Data

We can use the dictionaries and lists we already know:

```
questions = [
    {"id": 1, "text": "Is the monitor switched on?"},
    {"id": 2, "text": "Is the monitor cable connected?"}
]

recommendations = [
    {"id": 1, "text": "Switch on the monitor"},
    {"id": 2, "text": "Check the monitor cable"}
]
```

Each **ID** gives us a way to refer to a particular question or recommendation.

Representing Our Rule as Data

Our rule can also be a dictionary:

```
rule = {
    "question_id": 1,
    "expected_answer": "no",
    "recommendation_id": 1
}
```

IF the answer to **question 1** is **“no”**, THEN suggest **recommendation 1**.

We are only storing the rule today. We will write code to check rules later.

What If the Screen Is Still Blank?

Suppose the user answers:

“Yes, the monitor is switched on — but the screen is still blank.”

Our first rule doesn’t help. We need more evidence.

- Is the monitor cable connected?
- Is the computer itself switched on?
- Is the monitor set to the correct input?

What If the Screen Is Still Blank?

Suppose the user answers:

“Yes, the monitor is switched on — but the screen is still blank.”

Our first rule doesn’t help. We need more evidence.

- Is the monitor cable connected?
- Is the computer itself switched on?
- Is the monitor set to the correct input?

Looking ahead: A more useful rule may need to check **several answers together**.

Questions gather evidence. Rules use the evidence. Recommendations provide advice.

The Bigger Picture

Over the coming weeks, we'll develop our PC Troubleshooter step by step.

Week 6: Lists, dictionaries, functions and CRUD

Next: Save our questions and rules in JSON files

Later: Use classes and objects, and build a more capable rule engine

> Our goal: change what the system knows without rewriting all its decision-making code.